

Evaluating Expressions

Executing code

The simplest, *shortest* path to executing code is through the syntax tree itself

Since our parser currently only supports expressions. No statements, no variables, no functions, no control flow,

All our interpreter will do is go through those expressions, and **generate a value**

This raises two questions

1. what kind of value
2. how do we organize the code

Representing Values

In lox, a *value* is created by **literals**, which are computed as *expressions*, and then stored in *variables*

And given that lox is a *dynamically typed language* and java is **not**, we need to bridge that gap somehow

```
1  java.lang.Object // the root of the class hierarchy in Java
```

In places where the interpreter needs to store a value, we use the `Object` type

Which we then convert during runtime to `number` or `string` or `bool`

Note

The code in the next slides expect a starting point of a working parser

You can clone the code available here

github.com/sininen-blue/alaan_cs370

From the commit `8385e1e7b5bf` `6.4 wiring up the parser`

Which is the latest commit as of writing this `March 11 2026`

Evaluating Expressions

For every type of expression we can parse, we need some code to define how it's actually evaluated

We'll do this by implementing another *Visitor* pattern

```
1  lox/Interpreter.java
2  create new file
```

```
1  package com.craftinginterpreters.lox;
2
3  class Interpreter implements Expr.Visitor<Object> {
4  }
```

So we declare a *visitor*, which returns an object, and all we need to do is define the *visit* methods for each class

Literals

The *Leaves* of our abstract syntax tree, the smallest bits of code in Lox, are **literals**

Literals are **almost** values, but not quite.

They **evaluate** to values, but they aren't values themselves.

```
1  lox/Interpreter.java
2  in class Interpreter
```

```
1  @Override
2  public Object visitLiteralExpr(Expr.Literal expr) {
3      return expr.value;
4  }
```

Parenthesis

The next simplest node to evaluate is the **grouping** expression

Remember that we defined the grouping expression as a single sub-expression wrapped in parentheses

```
1  Grouping(Expr expression) {  
2      this.expression = expression;  
3  }
```

So all we have to do is evaluate the expression, and return it

```
1  lox/Interpreter.java  
2  in class Interpreter
```

```
1  @Override  
2  public Object visitGroupingExpr(Expr.Grouping expr) {  
3      return evaluate(expr.expression);  
4  }  
5  
6  private Object evaluate(Expr expr) {
```

Unary

Like grouping, we have a single sub-expression that needs to be evaluated

```
1  lox/Interpreter.java
2  after visitLiteralExpr()
```

```
1  @Override
2  public Object visitUnaryExpr(Expr.Unary expr) {
3      Object right = evaluate(expr.right);
4
5      switch (expr.operator.type) {
6          case MINUS:
7              return -(double)right;
8          case BANG:
9              return !isTruthy(right);
10     }
11     // Unreachable.
12
13     return null;
14 }
```

Post Order Traversal

Our evaluation recursively traverses the tree. We can't evaluate the unary operator itself until **after** we evaluate its operand

That means our interpreter is doing a **post-order** traversal

Each node evaluates its children before doing its own work

Truthiness

What happens when something that **isn't** a boolean, is used in a logical operation

One way of solving this is to **error**, but most dynamically typed languages don't do this

```
1  const x = 5;  
2  if ( x ) console.log("x is true");
```

This is called *truthiness*, it's arbitrary, and every language has its own rules for it

Lox follows the same rules **Ruby** does, which is that **false** and **nil** are falsey, and everything else is truthy

```
1  lox/interpreter.java  
2  in after visitUnaryExpr()
```

```
1  private boolean isTruthy(Object object) {  
2      if (object == null) return false;  
3      if (object instanceof Boolean) return (boolean)object;  
4      return true;  
5  }
```

Binary

There's a few binary operations, but they all follow the same pattern

```
1  lox/interpreter.java
2  in class Interpreter
```

```
1  @Override
2  public Object visitBinaryExpr(Expr.Binary expr) {
3      Object left = evaluate(expr.left);
4      Object right = evaluate(expr.right);
5
6      switch (expr.operator.type) {
7          case MINUS:
8              return (double)left - (double)right;
9          case SLASH:
10             return (double)left / (double)right;
11          case STAR:
12             return (double)left * (double)right;
13      }
14
15      return null;
```

Addition

The addition operator is special in lox, because it supports two different types, numbers and strings

```
1  lox/interpreter.java
2  in switch statement in visitBinaryExpr()
```

```
1      case PLUS:
2          if (left instanceof Double && right instanceof Double) {
3              return (double)left + (double)right;
4          }
5
6          if (left instanceof String && right instanceof String) {
7              return (String)left + (String)right;
8          }
9          break;
```

Comparison

The comparison operators are fairly straightforward, but they only work on numbers

```
1  lox/interpreter.java
2  in switch statement in visitBinaryExpr()
```

```
1      switch (expr.operator.type) {
2          case GREATER:
3              return (double)left > (double)right;
4          case GREATER_EQUAL:
5              return (double)left ≥ (double)right;
6          case LESS:
7              return (double)left < (double)right;
8          case LESS_EQUAL:
9              return (double)left ≤ (double)right;
10     }
```

Equality

The last pair are

```
1  lox/interpreter.java
2  in switch statement in visitBinaryExpr()
```

```
1      case BANG_EQUAL:
2          return !isEqual(left, right);
3      case EQUAL_EQUAL:
4          return isEqual(left, right);
```

```
1  lox/interpreter.java
2  after isTruthy()
```

```
1  private boolean isEqual(Object a, Object b) {
2      if (a == null && b == null) return true;
3      if (a == null) return false;
4
5      return a.equals(b);
6  }
```

This is a weird corner case in java